# Plan de Implementación — SafeMine AI

Plataforma Integral de Acreditación Minera, Validación Documental con IA y Control de Acceso Físico mediante Detección de EPPs en Tiempo Real.

> IMPORTANT
>
> **Actualización de SDK y Modelo Gemini**: El SDK legado `google-generativeai` y el modelo `gemini-2.5-flash` están deprecados. En su lugar, utilizaremos el SDK oficial actual **`google-genai` ( `>= 2.30.1` )** con `client.interactions.create` y el modelo multimodal **`gemini-3.8-flash`** con salida estructurada JSON ( `response_format` ), manteniendo además el interruptor de simulación ( `USE_MOCK_AI=true` ) en `.env` para pruebas locales inmediatas sin consumir cuota.

## 1. Arquitectura General del Sistema

## 2. Estructura de Carpetas del Proyecto

```
text
SafeMine/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                # Punto de entrada FastAPI, CORS, WebSockets y auto-
seed
│   │   ├── config.py              # Variables de entorno e interruptores Mock/Real
│   │   ├── database.py            # Configuración SQLAlchemy (SQLite / PostgreSQL RDS)
│   │   ├── models.py              # Modelos ORM (User, Proposal, WorkerProfile,
EvaluationReport, AccessLog)
│   │   ├── schemas.py             # Esquemas Pydantic (Request/Response)
│   │   ├── seed.py                # Datos semilla: 1 Minera, 1 Proveedor, 1 Propuesta con
10 trabajadores
│   │   ├── routers/
│   │   │   ├── auth.py            # Login por rol/credenciales y registro de proveedores
│   │   │   ├── minera.py          # Crear convocatorias, aceptar propuestas, calificar y
emitir reporte
│   │   │   ├── proveedor.py       # Ver convocatorias, postular cuadrilla de 10, ver
credenciales y reportes
│   │   │   ├── trabajador.py      # Examen (>=80%), subida de constancia médica (Gemini
IA), Fotocheck QR
│   │   │   └── kiosco.py          # Fase 1 (Validar QR) + Fase 2 (AWS Rekognition EPPs) +
WebSockets
│   │   └── services/
│   │       ├── gemini_service.py  # Validación documental con google-genai (gemini-3.8-
flash) + modo Mock
│   │       ├── aws_service.py     # Detección de EPPs con boto3 Rekognition + S3 + modo
Mock
│   │       └── ws_manager.py      # Gestor de conexiones WebSocket (Kiosco UI + Arduino
Bridge)
│   ├── uploads/                   # Almacenamiento local de constancias médicas (fallback
de S3)
│   ├── requirements.txt
│   ├── .env.example
│   └── .env
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── layout.tsx         # Layout principal con barra de navegación rápida entre
roles y kiosco
│   │   │   ├── page.tsx           # Portal de Inicio de Sesión / Selector rápido de Demo
por Rol
```

```
│   │   │   ├── dashboard/
│   │   │   │   ├── minera/page.tsx     # Módulo 1: Convocatorias, Aceptación, Tabla de Evaluación y PDF
│   │   │   │   ├── proveedor/page.tsx  # Módulo 2: Postulación de cuadrilla, Credenciales y Reportes
│   │   │   │   └── trabajador/page.tsx # Módulo 3: Inducción/Examen (>=80%), Constancia IA y Fotocheck QR
│   │   │   └── kiosco-camara/page.tsx  # Módulo 4: Máquina de estados Fase 1 (QR) -> Fase 2 (EPP + WS)
│   │   ├── components/             # Componentes reutilizables (Navbar, Badges, Modales, QR Card)
│   │   └── lib/
│   │       ├── api.ts              # Cliente HTTP y configuración de endpoints / WebSocket URL
│   │       └── pdfGenerator.ts     # Generador de Reporte Formal en PDF con jspdf y jspdf-autotable
│   ├── package.json
│   ├── .env.example
│   └── .env.local
└── iot-arduino/
    ├── arduino_bridge.py           # Cliente WebSocket en Python que retransmite 'V' (Verde) o 'R' (Rojo) por Serial
    ├── safemine_access_control/
    │   └── safemine_access_control.ino  # Firmware Arduino UNO / ESP32 (Pin 8 LED Verde, Pin 9 LED Rojo + Buzzer)
    ├── requirements.txt
    └── README.md
```

## 3. Modelo de Datos (`backend/app/models.py`)

| Entidad | Campos Principales | Propósito en el Flujo de Negocio |
|---|---|---|
| **`User`** | `id`, `email`, `username`, `password_hash`, `plain_password_hint`, `role` (`MINERA`, `PROVEEDOR`, `TRABAJADOR`), `full_name`, `company_name` | Autenticación de los 3 roles. Guarda `plain_password_hint` para las cuentas autogeneradas de los trabajadores de modo que el Proveedor pueda visualizar y entregar sus credenciales. |
| **`Proposal`** | `id`, `title`, `description`, `quota` (ej. 10), `start_date`, `required_epps` (JSON: `["HEAD_COVER", "HAND_COVER", "FACE_COVER"]`), `status` (`ABIERTA`, `POSTULADA`, `ACEPTADA`), `provider id`, `report_sent` (bool), `report_sent_at` | Convocatorias creadas por la Minera y postuladas por un Proveedor. Al pasar a `ACEPTADA`, genera automáticamente las cuentas de los trabajadores. |
| **`WorkerProfile`** | `id`, `user_id`, `proposal_id`, `first_name`, `last_name`, `dni`, `email`, `position`, `exam_score` (int), `exam_passed` (bool), `medical_cert_url`, `medical_cert_status` (`PENDIENTE`, `VALIDADA_POR_IA`, `RECHAZADA`), `medical_ai_feedback`, `is_authorized` (bool), `qr_token`, `bad_conduct` (bool), `poor_performance` (bool), `missing_epp` (bool), `epp_incidents_count` (int), `evaluation_notes` | Perfil integral del trabajador: controla los 3 pasos de acreditación, almacena los 3 checkboxes de evaluación (`bad_conduct`, `poor_performance`, `missing_epp`) y el contador automático de faltas de EPP en cámara. |
| **`AccessLog`** | `id`, `worker_id`, `timestamp`, `phase` (`QR_SCAN`, `EPP_CHECK`), `status` (`VERDE`, `ROJO`), `missing_items` (JSON), `details` | Historial en vivo de eventos en garita para auditoría de la Minera y sincronización en tiempo real. |

## 4. Detalle de Módulos y Reglas de Negocio

### Módulo 1: Rol Minera (`/dashboard/minera`)

1. **Crear Convocatorias**: Formulario con título, descripción, cupo de trabajadores (por defecto 10), fecha de inicio y selección de EPPs obligatorios (`Casco / HEAD_COVER`, `Guantes / HAND_COVER`, `Protección Facial / Chaleco / FACE_COVER`).
2. **Bandeja de Propuestas**: Lista de propuestas con los trabajadores postulados por el Proveedor.
    - Acción **"Aceptar Propuesta"** (`POST /api/minera/proposals/{id}/accept`): Cambia el estado a `ACEPTADA` y genera automáticamente las cuentas de usuario (`User`) para los 10

trabajadores con credenciales únicas (ej. usuario basado en DNI/correo y contraseña generada).

3. **Calificación de Trabajadores y Reporte al Proveedor**:
    - Tabla interactiva de los trabajadores de la propuesta aceptada con estado de acreditación, contador de faltas en cámara y los **3 Checkboxes**:
        - `[ ] Mala conducta` (`bad_conduct`)
        - `[ ] Mal trabajo / Bajo desempeño` (`poor_performance`)
        - `[ ] No lleva sus EPPs` (`missing_epp` — editable manualmente y **marcado automáticamente por el backend** cuando el trabajador falla en la cámara de garita).
    - Campo de observaciones por trabajador y botón **"Guardar Calificaciones"** (`PUT /api/minera/proposals/{id}/evaluations`).
    - Botón **"Descargar Reporte PDF y Enviar al Proveedor"**: Genera el PDF con `jspdf` + `jspdf-autotable` separando el resumen ejecutivo de cumplimientos vs. incumplimientos, y marca `report_sent = True` (`POST /api/minera/proposals/{id}/send-report`) para que aparezca inmediatamente en la bandeja del Proveedor.

## Módulo 2: Rol Proveedor (`/dashboard/proveedor`)

1. **Registro e Inicio de Sesión**: Creación de cuenta de contratista o acceso rápido con la cuenta semilla.
2. **Postular Cuadrilla a Propuestas Abiertas**:
    - Visualiza convocatorias abiertas y permite cargar la nómina de los trabajadores solicitados (Nombres, Apellidos, DNI, Correo, Cargo), con botón de autocompletar plantilla rápida para pruebas ágiles.
3. **Credenciales Generadas y Seguimiento**:
    - Cuando la Minera acepta la propuesta, muestra la tabla de credenciales generadas (`Usuario / DNI`, `Contraseña temporal`) y el progreso en tiempo real de cada trabajador (`Examen >= 80%`, `Constancia Médica IA`, `Estado: AUTORIZADO / PENDIENTE`).
4. **Bandeja de Reportes de Evaluación**:
    - Visualiza los reportes enviados por la Minera con el detalle de quiénes tuvieron desempeño óptimo y quiénes registraron *Mala conducta*, *Mal trabajo* o *No lleva sus EPPs* (incluyendo la opción de descargar el PDF desde el portal del proveedor).

## Módulo 3: Rol Trabajador ( `/dashboard/trabajador` )

1. **Paso 1 — Capacitación e Inducción + Examen**:
    - Contenido de inducción sobre seguridad minera e IPERC / uso de EPPs.
    - Examen interactivo de 5 preguntas (20 puntos c/u).
    - **Regla estricta**: Requiere `>= 80/100` para aprobar ( `exam_passed = True` ). Si obtiene `< 80` , permite reintentar inmediatamente; al obtener `>= 80` , desbloquea el Paso 2.
2. **Paso 2 — Subida de Constancia Médica Validada por IA ( `gemini-3.8-flash` )**:
    - Sube archivo PDF, JPG o PNG ( `POST /api/trabajador/{worker_id}/medical-cert` ).
    - El servicio `gemini_service.py` procesa el documento con `google-genai` ( `gemini-3.8-flash` ) y `response_format` JSON verificando:
        - `is_valid_medical_certificate` : boolean
        - `worker_name_detected` : string
        - `aptitude_result` : `"APTO"` | `"NO APTO"` | `"NO DETECTADO"`
        - `summary` : explicación breve de la validación.
    - Incluye selector/modo simulación para probar tanto un certificado **"APTO"** como un documento inválido/falso sin requerir API Key activa.
3. **Paso 3 — Pase de Autorización y Fotocheck QR**:
    - Al cumplir `exam_score >= 80` y `medical_cert_status == "VALIDADA_POR_IA"` , el estado cambia automáticamente a `AUTORIZADO PARA IR A MINA` ( `is_authorized = True` ).
    - Renderiza el **Fotocheck Digital** con `qrcode.react` codificando `{"worker_id": X, "dni": "...", "app": "SAFEMINE"}` .

## Módulo 4: Kiosco / Cámara en Vivo en Garita ( `/kiosco-camara` ) + Rekognition + Arduino

- **Fase 1 (Escaneo de QR)**: Usa `html5-qrcode` en la cámara (y también ofrece un selector rápido de trabajadores en pantalla para probar desde PC sin tener que apuntar el celular si se desea). Verifica `POST /api/vision/verify-qr` .
- **Fase 2 (Detección de EPPs)**: Captura frame en vivo de la cámara ( `<video>` + `<canvas>` ) y lo envía a `POST /api/vision/verify-epp` junto con el `worker_id` recién escaneado.
    - En modo AWS Real ( `USE_MOCK_AWS=false` ), invoca `rekognition.detect_protective_equipment(Image={'Bytes': ...})` validando `HEAD_COVER` , `HAND_COVER` y `FACE_COVER` .

- Incluye botones de simulación directa en pantalla (**"Simular Detección CON EPPs"** y **"Simular Detección SIN EPPs"**) para pruebas locales instantáneas.
- Si falta algún EPP: emite `{"status": "ROJO", "worker_id": X, "faltante": [...]}` por WebSocket, **marca automáticamente** `missing_epp = True` **(** `no_lleva_epps` **) y suma** `+1` **a** `epp_incidents_count` en el registro del trabajador.

## 5. Datos Semilla (Seed Data) Incluidos al Iniciar

Al levantar el backend por primera vez, `seed.py` poblará automáticamente:

1. **Cuenta Minera**: `minera@safemine.pe` / `minera123` (Rol: `MINERA`).
2. **Cuenta Proveedor**: `proveedor@contratistasperu.pe` / `proveedor123` (Rol: `PROVEEDOR` — *Contratistas Mineros del Sur S.A.C.*).
3. **Propuesta #1 (Lista para Aceptar)**: "*Mantenimiento Preventivo de Planta Concentradora - Guardia A*" con cupo de **10 trabajadores ya postulados por el Proveedor**, lista para que la Minera haga clic en **"Aceptar Propuesta"** y vea cómo se crean las 10 cuentas automáticamente.
4. **Propuesta #2 (Ya Aceptada con Trabajadores de Prueba en Distintas Etapas)**:
   - Trabajador 1 ( `Carlos Mendoza` , DNI `45891021` ): **Ya Autorizado** ( `Examen: 100/100` , `Constancia: VALIDADA_POR_IA` ) listo para probar inmediatamente su QR en `/kiosco-camara` .
   - Trabajador 2 ( `Jorge Quispe` , DNI `46102938` ): En Paso 1 (pendiente de rendir examen y subir constancia) para probar el flujo de trabajador desde cero.
   - Trabajadores adicionales activos para probar los checkboxes de evaluación y la descarga/envío del Reporte PDF al Proveedor.

## 6. Configuración de Variables de Entorno ( `.env.example` )

Crearemos `backend/.env.example` y `backend/.env` con interruptores claros:

```ini
# Base de Datos (SQLite local por defecto; descomentar PostgreSQL para AWS RDS)
DATABASE_URL=sqlite:///./safemine.db
# DATABASE_URL=postgresql://postgres:password@safemine-rds.xxxx.us-east-1.rds.amazonaws.com:5432/safemine

# Seguridad JWT
JWT_SECRET_KEY=safemine-super-secret-key-2026

# Interruptores de Simulación (true = modo simulación local sin gastar créditos, false = usa APIs reales)
USE_MOCK_AI=true
USE_MOCK_AWS=true

# Google Gemini API (SDK google-genai >= 2.30.1, modelo gemini-3.8-flash)
GEMINI_API_KEY=tu_gemini_api_key_aqui
GEMINI_MODEL=gemini-3.8-flash

# Credenciales AWS (Rekognition para EPPs y S3 para documentos)
AWS_ACCESS_KEY_ID=tu_aws_access_key_id
AWS_SECRET_ACCESS_KEY=tu_aws_secret_access_key
AWS_REGION=us-east-1
AWS_S3_BUCKET=safemine-documents-bucket
min_epp_confidence=80.0
```

## 7. Secuencia de Ejecución del MVP

1. **Inicializar** `/backend`: Crear entorno e instalar `fastapi`, `uvicorn[standard]`, `sqlalchemy`, `pydantic`, `python-multipart`, `google-genai`, `boto3`, `python-dotenv`, `websockets`. Construir modelos, servicios, routers y seed data.
2. **Inicializar** `/frontend`: Configurar proyecto Next.js (App Router + TypeScript + Tailwind CSS) e instalar `lucide-react`, `html5-qrcode`, `qrcode.react`, `jspdf`, `jspdf-autotable`. Construir los 4 módulos con diseño industrial minero oscuro/ámbar de alto contraste.
3. **Inicializar** `/iot-arduino`: Crear `arduino_bridge.py`, `requirements.txt` (`websockets`, `pyserial`) y el sketch `.ino` para Arduino UNO/ESP32 (Pin 8 LED Verde, Pin 9 LED Rojo + Buzzer).
4. **Levantar Servidores y Verificar**: Iniciar FastAPI en `http://localhost:8000` y Next.js en `http://localhost:3000` para probar el flujo completo de principio a fin.